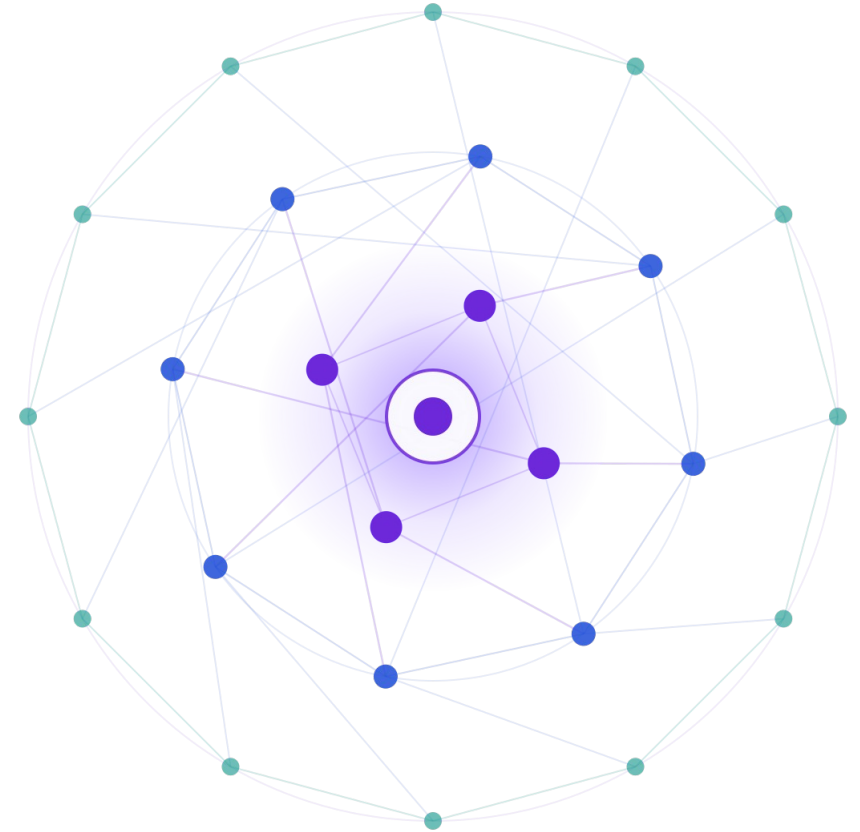


Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID – 1 (Agentic Code Intelligence)
 - Team Name – Trace
 - College Name – Thapar Institute of Engineering and Technology, Patiala
 - Member – Abhavya Jha – ajha2_be24@thapar.edu
-
- Submission GitHub link – <https://github.com/Ajha2005/samsung-prism>



Theme

Theme 1 — Agentic Code Intelligence

Given a natural-language query and a library of code snippets, return the snippets ranked by relevance — pure retrieval, no generation or explanation.

Screening ranks teams on MTEB's NDCG@10 and MRR over the ColR AppsRetrieval test split; the top teams then go to a hands-on review of the code, deck and demo.

THE SCOREBOARD · SUBMITTED: e5-base-v2

NDCG@10 = 0.1151

MRR@10 = 0.0986 · +73.9% over baseline

THE DISCIPLINE THAT PRODUCED IT

10

configs measured

8

dropped

1

kept: e5-base

78

offline tests

1

CPU-only, minimal GPU

Innovation has to come from representation, not scale

2

No LLM in the ranking loop

Retrieval must beat generation on speed on a thousand-snippet corpus

3

Cross-version retrieval (P1)

The index must rebuild cheaply as code changes, by design from day one

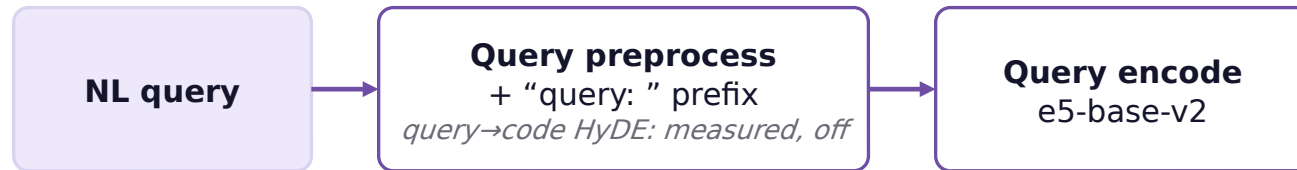
Existing Solutions & Gaps

WHAT TEAMS USUALLY TRY	WHERE IT BREAKS DOWN	WHAT WE BUILT & MEASURED
1 Dense embedding + cosine similarity with an off-the-shelf text model	NL and code sit in different “languages” — behavioral queries blur	Code-flavored HyDE: compile query → deterministic code sketch, then embed code-vs-code
2 LLM re-ranking on top of a first-pass retriever	Corpus in the thousands → per-query LLM ranking slower than retrieval itself	No LLM in the ranking loop; any LLM use is offline pre-processing, cached
3 One raw-text view per snippet, straight into the encoder	Raw text blurs the structure and identifiers usage & structural queries need	Multi-view snippet embedding: raw + AST-normalized + identifier bag, fused
4 A single index built once for one code version	Near-identical versions become indistinguishable; every change rebuilds all	Content-addressed versioned index + stable-core / version-delta representation

Measured: rows 1 and 3 (HyDE, multi-view) lost and are off; row 4's version-delta ties naive retrieval; e5-base-v2 is what moved the score.

Our Solutions & Architecture Diagram

QUERY PATH

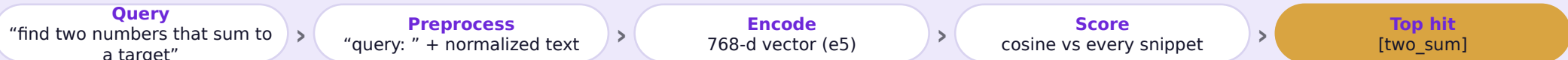


SNIPPET PATH



Submitted: e5-base-v2, 512-token window, CPU-only. Grey items are built and switchable by config, but lost on the real split and are off.

EXAMPLE TRACE — one real query through the pipeline



Demo & Product Walkthrough



```
$ python -m prism.cli demo --backend hashing  
[prism-demo] embedding backend: offline HashingBackend
```

Q: find two numbers that sum to a target

1. `[two_sum]` score=0.3810
2. `[binary_search]` score=0.1483
3. `[lru_cache_manual]` score=0.0785

~42 ms

per query, end to end
e5-base-v2 on CPU

84 min

full real eval, e5-base-v2
12,530 texts on a free Kaggle CPU

3 more behavioural queries, top-1 hits on the same corpus

Q “check if a string is a palindrome”

► **is_palindrome**

score 0.44

Q “shortest path in an unweighted graph”

► **bfs**

score 0.15

Q “count word frequencies in a text”

► **word_count**

score 0.48

Offline demo (hashing backend, no downloads): same pipeline and ranking code as the leaderboard run — the submitted model swaps in by config.

Tools and tech stack used

1

Core ML & evaluation

MTEB is the same library the leaderboard runs; picking it removes any doubt about metric parity.

Python 3.11 · PyTorch (CPU) · sentence-transformers · MTEB (official scorer)

2

Retrieval signals

Stdlib `ast` avoids a third-party parser dependency — the eval box does not need to install one.

NumPy · rank-bm25 (sparse/BM25) · Python `ast` (structural normalization)

3

Reproducibility & infra

The default `docker run` runs the whole test suite and demo with no HuggingFace round-trip needed.

Docker · pytest — 78 offline tests · Makefile: one command per step

BY THE NUMBERS

78

offline tests

10

configs measured

1

docker run

~42 ms

per query, e5 on CPU

100%

CPU inference

Impact & Use case

1

Faster onboarding

For: new engineers

Search unfamiliar code by describing behavior, not memorized function names.

EXAMPLE QUERY

"where do we retry a failed payment?"

► *Days-to-productivity → hours.*

2

Agentic coding backend

For: AI coding agents

A fast retrieval step an agent calls before it generates or edits code.

RETRIEVAL BUDGET

one encoder pass per query, no LLM in the loop.

► *Retrieval faster than generation, by design.*

3

Code review & audit

For: reviewers / security

Locate every place a pattern or behavior appears — not just a grep match.

EXAMPLE QUERY

"all endpoints that write user email unmasked"

► *Finds behavioral clones grep misses.*

4

Legacy modernization

For: platform teams (P1)

Search & compare behavior across many historical versions of the same code.

EXAMPLE QUERY

"auth check pre-v3.2 vs post-v3.2"

► *Cross-version rebuilds cheap by design.*

Innovation highlights, results and limitations

Innovation highlights

Built and rigorously measured — kept only where the number agreed (see results →)

- Query→code compilation (code-flavored HyDE), LLM-free at eval time
- Multi-view snippet embedding: raw + AST-normalized + identifier bag
- Keyword front-loading: salient signals survive a 256-token tokenizer cap
- Stable-core + version-delta representation for cross-version retrieval
- Content-addressed versioned index — rebuilds only what changed

THE DISCIPLINE THAT PRODUCED THAT NUMBER

Propose
one config

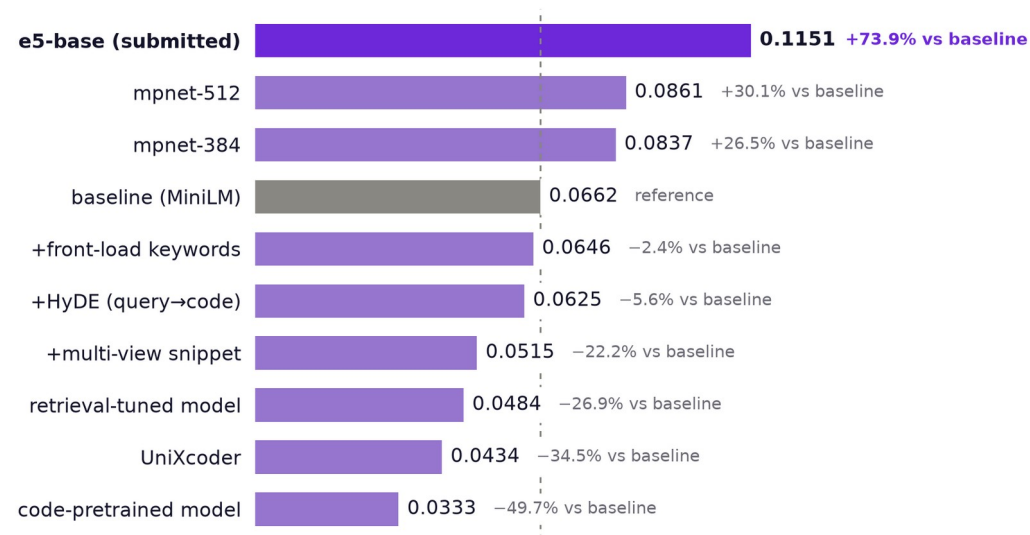
Measure
NDCG@10 on
real split

Compare
vs live baseline

Keep or drop
no drift, ever

NDCG@10, real CoIR AppsRetrieval split · 10 configs, 1 kept

Submitted: e5-base-v2 — MRR@10 0.0986 (baseline 0.0561).



Limitations — stated up front

- CPU-only by design; larger models (e.g. e5-large, ~3× CPU) untested
- HyDE, multi-view, front-load not yet re-measured on top of e5
- Version-delta ties naive retrieval, even on e5: no cross-version gain yet

What's next

WHAT THE ABLATION TAUGHT US

Ten configs on the real split. The winner, e5-base-v2, is the same size as mpnet but trained for query→passage retrieval: +73.9% over baseline, +33.7% over mpnet. Every code-specialized swap (CodeSearchNet, UniXcoder) lost — the roadmap follows that evidence.

1

Re-test on e5

HIGH

HyDE, multi-view and front-loading lost on MiniLM; the stronger e5 encoder may use them differently.

2

Scale the winner

MED

e5-large-v2 is the natural next test, at roughly 3× the CPU time.

3

Cross-version gain

MED

Version-delta ties naive retrieval so far, even on e5. Make it win, then test on real multi-version repos.

4

Publish the findings

LONG

Cross-version code retrieval is under-explored; write up these ablation findings as a short paper.

Every step above is measured the same way as everything before it — one change, one number, keep or revert.

Brownie points slide(differentiation)



Verified, not assumed

Proven end-to-end, and checked against a published number.

EVIDENCE

Our e5-base-v2 run (0.1151) matches the published CoIR score for that model (11.5); 6 tests lock the file shape.



Discipline over guesswork

Every feature logged config → NDCG@10 → kept/dropped, every time.

EVIDENCE

docs/ablation_log.md — 10 real experiments, 8 dropped, 1 kept (+73.9%); every row reproducible from the repo.



Versioned from day one

Content-addressed index; cross-version retrieval never bolted on.

EVIDENCE

prism/index/versioned.py — a new version re-encodes only the snippets that changed (1 of 18 in the demo).



Honest about limits

Limitations reported next to results — credibility over overselling.

EVIDENCE

docs/ablation_log.md — every failure named up front; a code-model swap losing 50% is documented, not hidden.

BY THE NUMBERS

10

real experiments

8

dropped, still in repo

78

offline tests

0

mocks in leaderboard eval

1

reproducible docker run

Checklist- Updated on Public GitHub

SUBMISSION CHECKLIST



Working prototype code

Public GitHub repo, tag PRISM_GENAI_HACKATHON_Y2026
github.com/Ajha2005/samsung-prism



README with reproducible setup

Docker one-command run + full CLI
`docker build -t prism-code-search . → docker run`



Demo video (≤ 5 min)

Recorded and linked in the submission form
[YouTube](#) / [Drive link](#) – see form



Presentation file (PPT + PDF)

This deck — 12 slides, speaker notes on every content slide
[Thapar_Trace_Submission_ppt.pptx](#) / [.pdf](#)



Leaderboard results JSON

Real CoIR AppsRetrieval, produced by `mteb.evaluate()`
[results/appsretrieval_results.json](#) (attached to release)



Ablation record

10 configs measured, 8 dropped, e5-base kept (+73.9%)
[docs/ablation_log.md](#)

SUBMISSION AT A GLANCE

0.1151

NDCG@10 · CoIR AppsRetrieval · MTEB

VERIFIED

10

real ablations

+73.9%

over baseline

78

offline tests

100%

CPU inference

Tag · PRISM_GENAI_HACKATHON_Y2026

Thank you

Team Trace · Thapar Institute of Engineering and Technology

Abhavya Jha

ajha2_be24@thapar.edu

github.com/Ajha2005/samsung-prism